



REC-CIS

Correct

Marked out of 1.00

Flag question

You are a bank account holder. Initially you have 1 rupee in your account, and you want exactly  $N$  rupees in your account. You wrote two hacks, first hack can multiply the amount of money you own by 10, while the second can multiply it by 20. These hacks can be used any number of time. Can you achieve the desired amount  $N$  using these hacks.

**Constraints:** $1 \leq T \leq 100$  $1 \leq N \leq 10^{12}$ **Input**

The test case contains a single integer  $N$

**Output**

For each test case, print a single line containing the string "1" if you can make exactly  $N$  rupees or "0" otherwise

SAMPLE INPUT

1

SAMPLE OUTPUT

1

SAMPLE INPUT

2

SAMPLE OUTPUT

0

**Answer:** (penalty regime: 0 %)

Reset answer

```

1 //
2 // * Complete the 'myFunc' function below.
3 // *
4 // * The function is expected to return an
5 // * The function accepts INTEGER n as para
6 // */
7
8 int myFunc(int n)
9 {
10     return n==1 || n%10==0;
11 }
12

```

|   | Test                      | Expected | Got |   |
|---|---------------------------|----------|-----|---|
| ✓ | printf("%d", myFunc(1))   | 1        | 1   | ✓ |
| ✓ | printf("%d", myFunc(2))   | 0        | 0   | ✓ |
| ✓ | printf("%d", myFunc(10))  | 1        | 1   | ✓ |
| ✓ | printf("%d", myFunc(25))  | 0        | 0   | ✓ |
| ✓ | printf("%d", myFunc(200)) | 1        | 1   | ✓ |

Passed all testcases ✓

Factoring  $n = 10$  results in  $\{1, 2, 5, 10\}$ . Return the  $p = 3^{10}$  factor, 5, as the answer.

#### Sample Case 1

##### Sample Input 1

STDIN Function

10 →  $n = 10$

5 →  $p = 5$

##### Sample Output 1

0

#### Explanation 1

Factoring  $n = 10$  results in  $\{1, 2, 5, 10\}$ . There are only 4 factors and  $p = 5$ , therefore 0 is returned as the answer.

#### Sample Case 2

##### Sample Input 2

STDIN Function

1 →  $n = 1$

1 →  $p = 1$

##### Sample Output 2

1

#### Explanation 2

Factoring  $n = 1$  results in  $\{1\}$ . The  $p = 1$ st factor of 1 is returned as the answer.

Answer: (penalty regime: 0 %)

Reset answer

```
1 //
2 * Complete the 'pthFactor' function below
3 *
4 * The function is expected to return a LONG_INTEGER
5 * The function accepts following parameters:
6 * 1. LONG_INTEGER n
7 * 2. LONG_INTEGER p
8 */
9
10 long pthFactor(long n, long p)
11 {
12     int count=0;
13     for(long i=1;i<=n;++i)
14     {
15         if(n%i==0)
16         {
17             count++;
18             if(count==p)
19             {
20                 return i;
21             }
22         }
23     }
24     return 0;
25 }
```

|   | Test                            | Expected | G |
|---|---------------------------------|----------|---|
| ✓ | printf("%ld", pthFactor(10, 3)) | 5        | 5 |
| ✓ | printf("%ld", pthFactor(10, 5)) | 0        | 0 |
| ✓ | printf("%ld", pthFactor(1, 1))  | 1        | 1 |

## Sample Case 0

## Sample Input 0

STDIN Function

32 → number = 32

## Sample Output 0

0

## Explanation 0

- Convert the decimal number 32 to binary number:  $32_{10} = (100000)_2$
- The value of the 4th index from the right in the binary representation is 0.

## Sample Case 1

## Sample Input 1

STDIN Function

77 → number = 77

## Sample Output 1

1

## Explanation 1

- Convert the decimal number 77 to binary number:  $77_{10} = (1001101)_2$
- The value of the 4th index from the right in the binary representation is 1.

Answer: (penalty regime: 0 %)

Reset answer

```

1 /*
2  * Complete the 'fourthBit' function below
3  *
4  * The function is expected to return an
5  * The function accepts INTEGER number as
6  */
7
8 int fourthBit(int number)
9 {
10     int binary[32];
11     int i=0;
12     while(number>0)
13     {
14         binary[i]=number%2;
15         number/=2;
16         i++;
17     }
18     if(i>=4)
19     {
20         return binary[3];
21     }
22     else
23         return 0;
24 }

```

|   | Test                        | Expected | Got |
|---|-----------------------------|----------|-----|
| ✓ | printf("%d", fourthBit(32)) | 0        | 0   |
| ✓ | printf("%d", fourthBit(77)) | 1        | 1   |

2

**Sample Output 0**

1

**Explanation 0**

If  $X = 10$  and  $N = 2$ , we need to find the number of ways that  $10$  can be represented as the sum of squares of unique numbers.

$$10 = 1^2 + 3^2$$

This is the only way in which  $10$  can be expressed as the sum of unique squares.

**Sample Input 1**

100

2

**Sample Output 1**

3

**Explanation 1**

$$100 = (10^2) = (6^2 + 8^2) = (1^2 + 3^2 + 4^2 + 5^2 + 7^2)$$

**Sample Input 2**

100

3

**Sample Output 2**

1

**Explanation 2**

$100$  can be expressed as the sum of the cubes of  $1, 2, 3, 4$  ( $1 + 8 + 27 + 64 = 100$ ). There is no other way to express  $100$  as the sum of cubes.

**Answer:** (penalty regime: 0 %)

Reset answer

```

1 //
2 * Complete the 'powerSum' function below
3 *
4 * The function is expected to return an
5 * The function accepts following paramet
6 * 1. INTEGER x
7 * 2. INTEGER n
8 */
9 #include<math.h>
10 int powerSum(int x, int n, int n)
11 {
12     int p=pow(n,n);
13     if (p==x)
14         return 1;
15     if (p>x)
16         return 0;
17     return powerSum(x-p,n+1,n)-powerSum(x
18 }

```

|   | Test                             | Expected |
|---|----------------------------------|----------|
| ✓ | printf("%d", powerSum(10, 1, 2)) | 1        |